

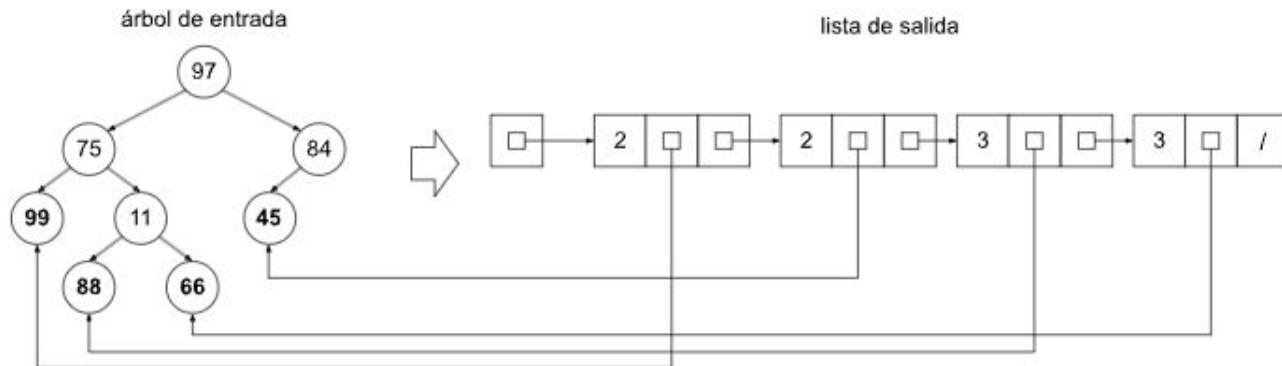
PARCIALITO ASM

PARCIALITO ASM

Utilizando las subrutinas `node_new` y `add_sort`, desarrollar la subrutina “`GET_LEAFS`”, y todas las necesarias, para que dado un árbol binario (**NO ordenado**), devuelva una lista (`nodo*`) que contenga los punteros a todas las hojas del árbol y se encuentre ordenada por el nivel de cada hoja.

```
PUSH <ptr a nodo árbol>
PUSH <nivel entero>
CALL node_new
ADD SP, 8
; Retorna en EAX un puntero nodo
nuevo de lista (nodo*).
```

```
PUSH <nodo* new>
PUSH <nodo** head>
CALL add_sort
ADD SP, 8
; Inserta ordenado en lista.
```



A	2 bytes	4 bytes	4 bytes
	Clave	ptr a izquierda	ptr a derecha
	Entero	Dir. Mem. tipo A	Dir. Mem. tipo A

L	2 bytes	4 bytes	4 bytes
	Nivel	ptr a nodo árbol	puntero a siguiente
	Entero	Dir. Mem. tipo A	Dir. Mem. tipo L

```
push <puntero a root>  
call get_leafs  
add sp, 4  
;retorna en eax el puntero a la lista
```

Definimos la especificación de la
invocación a subrutinas

```
push <puntero a root>  
call get_leafs  
add sp, 4  
;retorna en eax el puntero a la lista
```

```
push <nivel>  
push <doble puntero a header de lista>  
push <puntero a root>  
call _get_leafs  
add sp, 12
```

Definimos la especificación de la invocación a subrutinas

null equ -1

get_leafs: push bp
 mov bp, sp

Definimos la subrutina principal

mov sp, bp
pop bp
ret

```
push <puntero a root>
call get_leafs
add  sp, 4
;retorna en eax el puntero a la lista

push <nivel>
push <doble puntero a header de lista>
a root>
fs
```

null equ -1

get_leafs: push bp
 mov bp, sp
 sub sp, 4

Se reserva de espacio para la variable local que contendrá el puntero a la cabeza de la lista

mov sp, bp
pop bp
ret

```
push <puntero a root>
call get_leafs
add  sp, 4
;retorna en eax el puntero a la lista

push <nivel>
push <doble puntero a header de lista>
push <puntero a root>
fs
```

```
null      equ      -1
```

```
get_leafs:  push     bp
             mov      bp, sp
             sub      sp, 4

             mov      eax, bp
             sub      eax, 4
             mov      [eax], null
```

```
             mov      sp, bp
             pop      bp
             ret
```

```
push <puntero a root>
call get_leafs
add  sp, 4
;retorna en eax el puntero a la lista

push <nivel>
push <doble puntero a header de lista>
```

Se crea en EAX un doble puntero a la cabeza de la lista (para enviar a la subrutina recursiva como referencia)

Se aprovecha para referencia para inicializar la lista vacía.

```
null      equ      -1
```

```
get_leafs:  push    bp
            mov     bp, sp
            sub     sp, 4

            mov     eax, bp
            sub     eax, 4 ; doble puntero a header
            mov     [eax], null
```

```
            cmp     [bp+8], null
            jz      gl_end
```

Si el árbol está vacío...

... Convierte a EAX en un puntero simple,
en este caso tendrá NULL, es decir una
lista vacía.

```
gl_end:    mov     eax, [eax]

            mov     sp, bp
            pop     bp
            ret
```

```
push <puntero a root>
call get_leafs
add  sp, 4
;retorna en eax el puntero a la lista

push <nivel>
push <doble puntero a header de lista>
push <puntero a root>
call _get_leafs
add  sp, 12
```


null equ -1

```
get_leafs:  push    bp
             mov     bp, sp
             sub     sp, 4

             mov     eax, bp
             sub     eax, 4 ; doble puntero a header
             mov     [eax], null
```

```
             cmp     [bp+8], null
             jz      gl_end
```

```
             push    0
             push    eax
             push    [bp+8]
             call    _get_leafs
             add     sp, 12
```

```
gl_end:     mov     eax, [eax] ; a puntero simple
```

```
             mov     sp, bp
             pop     bp
             ret
```

```
push <puntero a root>
call get_leafs
add  sp, 4
;retorna en eax el puntero a la lista
```

```
push <nivel>
push <doble puntero a header de lista>
push <puntero a root>
call _get_leafs
add  sp, 12
```

Invoca la subrutina recursiva con los
parámetros inicializados:
Nivel en 0
nodo** en EAX
y [BP+8] con el puntero a la raíz del árbol

null equ -1

```
get_leafs:  push    bp
            mov     bp, sp
            sub     sp, 4

            mov     eax, bp
            sub     eax, 4 ; doble puntero a header
            mov     [eax], null
```

```
            cmp     [bp+8], null
            jz      gl_end
```

```
            push    0      ; nivel inicial
            push    eax
            push    [bp+8]
            call    _get_leafs
            add     sp, 12
```

gl_end: mov eax, [eax] ; a puntero simple

```
            mov     sp, bp
            pop     bp
            ret
```

```
push <puntero a root>
call get_leafs
add  sp, 4
;retorna en eax el puntero a la lista
```

```
push <nivel>
push <doble puntero a header de lista>
push <puntero a root>
call _get_leafs
add  sp, 12
```

```
left      equ      2
right     equ      6
```

```
_get_leafs: push    bp
             mov     bp, sp
```

Subrutina recursiva, sólo debería ser llamada desde “get_leafs”.

Estructura principal del árbol, no se agrega la clave porque en este ejercicio no se utiliza, pero no estaría de más:

```
clave      EQU      0
```

```
mov        sp, bp
pop        bp
ret
```

```
left    equ    2
right   equ    6
```

```
_get_leafs: push    bp
             mov     bp, sp
```

```
             push    ebx
```

```
             push    edx
```

```
             mov     ebx, [bp+8]
             mov     edx, [bp+12]; node**
```

En EBX el puntero a la raíz del árbol.

En EDX es doble puntero a la lista a completar

```
             pop     edx
```

```
             pop     ebx
```

```
             mov     sp, bp
             pop     bp
             ret
```

```
left    equ    2
right   equ    6
```

```
_get_leafs: push    bp
            mov     bp, sp
```

```
            push    ebx
```

```
            push    edx
```

```
            mov     ebx, [bp+8]
            mov     edx, [bp+12]; node**
```

```
            cmp     ebx, null
            jz      fin
            cmp     [ebx+left], null
            jz      test_der
```

Se vuelve a verificar que el árbol no esté vacío.
(No es necesario para el primer llamado, pero
puede serlo en una invocación recursiva).

Se comienza a deducir si el nodo es hoja,
primero preguntado por si la rama derecha está
vacía.

```
            pop     ebx
```

```
            mov     sp, bp
            pop     bp
            ret
```

```
left    equ    2
right   equ    6
```

```
_get_leafs: push    bp
            mov     bp, sp

            push    ebx

            push    edx

            mov     ebx, [bp+8]
            mov     edx, [bp+12]; node**
```

```
            cmp     ebx, null
            jz      fin
            cmp     [ebx+left], null
            jz      test_der
```

```
test_der: cmp     [ebx+right], null
            jz      agrega
```

Si la rama izquierda está vacía, consulta por la derecha.

```
            pop     edx
```

```
            pop     ebx
```

```
            mov     sp, bp
            pop     bp
            ret
```

left	equ	2	test_der:	cmp	[ebx+right], null
right	equ	6		jz	agrega

```

_get_leafs: push    bp
            mov     bp, sp
            push    eax
            push    ebx

            push    edx

```

Si ambas ramas están vacías, es una hoja y debe agregar el nodo a la lista.

```

mov     ebx, [bp+8]
mov     edx, [bp+12]; node**

```

```

agrega:  push    ebx      ;A*
          push    w[bp+18] ;short
          call    node_new
          add     sp, 8

```

Llama a “new_node” para crear el nodo, que devuelve en EAX el puntero al mismo.

```

cmp     ebx, null
jz      fin
cmp     [ebx+left], null
jz      test_der

```

```

          pop     edx

          pop     ebx
          pop     eax
          mov     sp, bp
          pop     bp
          ret

```

left	equ	2		test_der:	cmp	[ebx+right], null
right	equ	6			jz	agrega
_get_leafs:	push	bp				
	mov	bp, sp				
	push	eax				
	push	ebx				
	 push	 edx				
	 mov	 ebx, [bp+8]	agrega:	push	ebx	;A*
	mov	edx, [bp+12]; node**		push	w[bp+18]	;short
				call	node_new	
				add	sp, 8	
	 cmp	 ebx, null		push	eax	
	jz	fin		push	edx	
	cmp	[ebx+left], null		call	add_sort	
	jz	test_der		add	sp, 8	
				 pop	 edx	
				 pop	 ebx	
				pop	eax	
				mov	sp, bp	
				pop	bp	
				ret		

Utiliza EAX con el puntero al nuevo nodo y EDX donde tiene el doble puntero a la lista para invocar a "add_sort" que agrega el nodo a la lista.

left	equ	2	test_der:	cmp	[ebx+right], null
right	equ	6		jz	agrega

```

_get_leafs: push bp
            mov bp, sp
            push eax
            push ebx
            push ecx
            push edx

```

Se utiliza CX (2 bytes) para guardar el siguiente nivel y pasarlo como parámetro en la invocación recursiva.

```

            mov ebx, [bp+8]
            mov edx, [bp+12]; node**
            mov cx, w[bp+18]; short
            add cx, 1;

```

```

            cmp ebx, null
            jz fin
            cmp [ebx+left], null
            jz test_der

```

```

agrega:     push ebx           ;A*
            push w[bp+18]      ;short
            call node_new
            add sp, 8

```

```

            push eax
            push edx
            call add_sort
            add sp, 8

```

```

            pop edx
            pop ecx
            pop ebx
            pop eax
            mov sp, bp
            pop bp
            ret

```

left	equ	2	test_der:	cmp	[ebx+right], null
right	equ	6		jz	agrega

```

_get_leafs: push bp
            mov  bp, sp
            push eax
            push ebx
            push ecx
            push edx

```

```

            mov  ebx, [bp+8]
            mov  edx, [bp+12]; node**
            mov  cx, w[bp+18]; short
            add  cx, 1;

```

```

            cmp  ebx, null
            jz   fin
            cmp  [ebx+left], null
            jz   test_der

```

```

            push cx
            push edx
            push [ebx+left] ;izq
            call _get_leafs
            add  sp, 12

```

```

agrega:     push ebx      ;A*
            push w[bp+18] ;short
            call node_new
            add  sp, 8

```

```

            push eax
            push edx

```

Si el nodo actual tiene rama izquierda se llama por esa rama en forma recursiva.

```

            pop  ecx
            pop  ebx
            pop  eax
            mov  sp, bp
            pop  bp
            ret

```

left	equ	2	test_der:	cmp	[ebx+right], null
right	equ	6		jz	agrega

```

_get_leafs: push bp
            mov  bp, sp
            push eax
            push ebx
            push ecx
            push edx

```

a_der:

```

mov  ebx, [bp+8]
mov  edx, [bp+12]; node**
mov  cx, w[bp+18]; short
add  cx, 1;

```

```

agrega:    push ebx      ;A*
            push w[bp+18] ;short
            call node_new
            add  sp, 8

```

```

cmp  ebx, null
jz   fin
cmp  [ebx+left], null
jz   test_der

```

```

push  eax
push  edx
call  add_sort
add  sp, 8

```

```

push  cx
push  edx
push  [ebx+left] ;izq
call  _get_leafs
add  sp, 12

```

```

pop  edx
pop  ecx

```

jmp a_der

```

pop  bp
ret

```

Luego directamente se dirige a invocar por la rama derecha (ya se sabe que el nodo NO es una hoja).

left	equ	2	test_der:	cmp	[ebx+right], null
right	equ	6		jz	agrega

```

_get_leafs: push    bp
            mov     bp, sp
            push    eax

```

Se invoca por derecha. Es Aquí donde se puede estar enviando NULL si el nodo no tiene rama derecha...

```

a_der:      push    cx
            push    edx
            push    [ebx+right] ;der
            call    _get_leafs
            add     sp, 12

```

```

mov     ebx, [bp+8]
mov     edx, [bp+12]; node**
mov     cx, w[bp+18]; short
add     cx, 1;

```

```

agrega:     push    ebx      ;A*
            push    w[bp+18] ;short
            call    node_new
            add     sp, 8

```

```

cmp     ebx, null
jz      fin
cmp     [ebx+left], null
jz      test_der

```

... Y en ese caso tiene sentido consultar por árbol nulo.

```

call    add_sort
add     sp, 8

```

```

push    cx
push    edx
push    [ebx+left] ;izq
call    _get_leafs
add     sp, 12

jmp     a_der

```

```

pop     edx
pop     ecx
pop     ebx
pop     eax
mov     sp, bp
pop     bp
ret

```

left	equ	2	test_der:	cmp	[ebx+right], null
right	equ	6		jz	agrega

```

_get_leafs: push bp
            mov  bp, sp
            push eax
            push ebx
            push esp

```

```

a_der:      push cx
            push edx
            push [ebx+right] ;der
            call _get_leafs
            add  sp, 12
            jmp  fin

```

Luego de la invocación por derecha, al no ser una hoja no debe hacer nada más y debe saltar al fin.

```

mov  ebx, [bp+8]
mov  edx, [bp+12]; node**
mov  cx, w[bp+18]; short
add  cx, 1;

```

```

agrega:     push ebx ;A*
            push w[bp+18] ;short
            call node_new
            add  sp, 8

```

```

cmp  ebx, null
jz   fin
cmp  [ebx+left], null
jz   test_der

```

```

push  eax
push  edx
call  add_sort
add  sp, 8

```

```

push  cx
push  edx
push  [ebx+left] ;izq
call  _get_leafs
add  sp, 12
jmp  a_der

```

```

fin:        pop  edx
            pop  ecx
            pop  ebx
            pop  eax
            mov  sp, bp
            pop  bp
            ret

```

left	equ	2	test_der:	cmp	[ebx+right], null
right	equ	6		jz	agrega
_get_leafs:	push	bp	a_der:	push	cx
	mov	bp, sp		push	edx
	push	eax		push	[ebx+right] ;der
	push	ebx		call	_get_leafs
	push	ecx		add	sp, 12
	push	edx		jmp	fin
	mov	ebx, [bp+8]	agrega:	push	ebx ;A*
	mov	edx, [bp+12]; node**		push	w[bp+18] ;short
	mov	cx, w[bp+18]; short		call	node_new
	add	cx, 1;		add	sp, 8
	cmp	ebx, null		push	eax
	jz	fin		push	edx
	cmp	[ebx+left], null		call	add_sort
	jz	test_der		add	sp, 8
	push	cx	fin:	pop	edx
	push	edx		pop	ecx
	push	[ebx+left] ;izq		pop	ebx
	call	_get_leafs		pop	eax
	add	sp, 12		mov	sp, bp
	jmp	a_der		pop	bp
				ret	

left	equ	2	test_der:	cmp	[ebx+right], null
right	equ	6		jz	agrega
sig	equ	6			
_get_leafs:	push	bp	a_der:	push	cx
	mov	bp, sp		push	edx
	push	eax		push	[ebx+right] ;der
	push	ebx		call	_get_leafs
	push	ecx		add	sp, 12
	push	edx		jmp	fin
	mov	ebx, [bp+8]	agrega:	push	ebx ;A*
	mov	edx, [bp+12]; node**		push	w[bp+18] ;short
	mov	cx, w[bp+18]; short		call	node_new
	add	cx, 1;		add	sp, 8
	cmp	ebx, null			
	jz	fin		mov	[eax+sig], [edx]
	cmp	[ebx+left], null		mov	[edx], eax
	jz	test_der			
	push	cx	fin:	pop	edx
	push	edx		pop	ecx
	push	[ebx+left] ;izq		pop	ebx
	call	_get_leafs		pop	eax
	add	sp, 12		mov	sp, bp
	jmp	a_der		pop	bp
				ret	

Modificación para
insertar por delante
en la lista.